AI Basics: A Short Study Guide

Sample document for testing a RAG project

1. What is Artificial Intelligence?

Artificial intelligence, or AI, is the field of computer science that builds software able to perform tasks that normally need human intelligence. Examples include understanding language, recognizing objects in photos, making decisions, and translating between languages.

AI is an umbrella term. Machine learning, deep learning and generative AI are all parts of it. Early AI systems relied on hand-written rules, for example a chess program with thousands of rules coded by programmers. Modern systems mostly learn their behavior from data instead.

Common uses of AI today include voice assistants, spam filters, recommendation systems on video and shopping sites, medical image analysis, fraud detection in banking, and chatbots that answer questions in natural language.

2. What is Machine Learning?

Machine learning, or ML, is a branch of AI where a model learns patterns from data instead of following rules written by a programmer. The programmer provides examples, and the algorithm adjusts itself until it can make good predictions on new data.

There are three main types. In supervised learning, the data has correct answers attached, such as emails labeled spam or not spam, and the model learns to predict the label. In unsupervised learning, there are no labels and the model finds structure by itself, for example grouping similar customers together. In reinforcement learning, an agent learns by trial and error and receives rewards for good actions.

Two classic unsupervised and text techniques are KMeans clustering and TF-IDF. KMeans splits data points into a chosen number of groups, called clusters, so that points in the same group are close to each other. TF-IDF scores how important a word is to one document compared with a whole collection of documents, so common words like the or and get low scores and distinctive words get high scores.

A popular Python library for classic machine learning is scikit-learn. It includes ready-made tools for classification, regression, clustering and text processing.

3. Large Language Models and Generative AI

A large language model, or LLM, is a machine learning model trained on huge amounts of text so that it can understand and generate language. It works by predicting the next piece of text, called a token, again and again, which lets it write answers, summaries, code and more.

Generative AI means AI that creates new content, such as text, images, audio, video or code. LLMs are the text-generating part of generative AI. Well-known LLM families include Claude, GPT, Gemini and Llama.

LLMs have two important limits. First, they only know what was in their training data, so they do not know about private documents or recent events. Second, they can hallucinate, which means they sometimes produce confident answers that are wrong.

Small open models such as Llama 3.2 can run on a personal laptop using a tool called Ollama. They are slower and less capable than large cloud models, but they are free and private, which makes them good for learning.

4. Embeddings and Vector Databases

An embedding is a list of numbers that represents the meaning of a piece of text. A model turns each sentence into a vector, for example 384 numbers when using the all-MiniLM-L6-v2 model. Sentences with similar meaning get vectors that are close together, even if they share no words.

To compare two embeddings, we use cosine similarity, a score that is higher when two vectors point in a similar direction. This allows semantic search, which finds text by meaning. For instance, a search for automobile can find a sentence about a car, which plain keyword search would miss.

A vector database stores embeddings together with their original text and finds the closest ones to a query very quickly. Chroma is a free vector database that runs locally. Other examples are FAISS, Qdrant and Pinecone.

Before text is embedded it is usually split into chunks. A chunk size of around 500 characters with a small overlap is a common starting point. Small chunks give precise matches but may lose context, while large chunks keep context but make the match less precise.

5. Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation, or RAG, is a technique that lets an LLM answer questions using your own documents. It solves the problem that an LLM does not know your private data and can hallucinate.

RAG works in two phases. In the indexing phase, the documents are split into chunks, each chunk is converted into an embedding, and the embeddings are stored in a vector database. In the question phase, the user's question is embedded, the vector database returns the most similar chunks, and those chunks are sent to the LLM together with the question.

The LLM is instructed to answer only from the provided context. If the answer is not in the retrieved chunks, it should say that it could not find the answer in the document. This makes answers more trustworthy, and the system can show the source pages that were used.

A simple RAG project can be built with Python using Ollama for the LLM, sentence-transformers for embeddings, Chroma for the vector database, scikit-learn for machine learning features such as topic clustering, and Streamlit to turn the project into a web application.